



The Extra Block

Quick facilitation notes & answer key

Goal

Students fix a bug where a program overshoots the target because it has one extra block; they remove the extra step and run again to confirm. Altering existing code by deleting a step and describing how it fixes the outcome is C3.2.

Ontario curriculum (Grade 1 — C3 Coding)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential events.

C3.2 — read and alter existing code, including code that involves sequential events, and describe how changes to the code affect the outcomes.

Materials

The printed worksheet and a pencil. Optional: a floor number path 0-5 so students walk past the target and feel the extra step.

Run it (about 20 min)

1. I DO: run Program 1 — 'start 1, forward 2 to 3, forward 1 to 4... forward 1 to 5. We passed the target — there's an extra block.'
2. WE DO: Exercise 1 — Bit lands on 5. Exercise 2 — cross out one 'forward 1'; the program lands on 4.
3. YOU DO: Exercise 3 — Program 2 lands on 0; remove one 'back 1' so $0 \rightarrow 2 \rightarrow 1$.
4. Connect: 'Going too far can mean one extra block — take it out and check.'

Answer key (modelled by the run-gate)

Ex 1 — Program 1: $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$. Bit lands on 5, past the target 4.

Ex 2 — remove one 'forward 1': $1 \rightarrow 3 \rightarrow 4$. Bit lands on 4.

Ex 3 — Program 2: $0 \rightarrow \text{forward } 2 \rightarrow 2 \rightarrow \text{back } 1 \rightarrow 1 \rightarrow \text{back } 1 \rightarrow 0$. Remove one 'back 1': $0 \rightarrow 2 \rightarrow 1$. Bit lands on 1.

Watch for & differentiate

Common slip: removing the wrong kind of block — have students count how far past the target Bit went, then remove a step of that size.

Simplify: do Program 1 only; cross out the last block and re-run.

Extend: ask whether removing a different block could also reach the target, and test it.

Success indicator: the child removes one extra block in each program and lands Bit exactly on the target (4, then 1).